CSE557 · BIG DATA ANALYTICS · CIE-1 (30 MARKS · CO1–CO3)

Last-Minute Revision Sheet

Everything for CIE-1 on a few pages: definitions, lists, step orders, numbers, comparisons, memory tricks and the diagrams to draw.

Unit 1 · Digital data · Big Data · Analytics

Unit 2 · Hadoop · HDFS · MapReduce · YARN

Unit 3 · Spark architecture & installation

💡 HOW TO USE THIS SHEET

30 min before the exam: read every **bold** word and every 🧠 box.
2 hours before: read it all once, then redraw the **15 must-draw diagrams** (last pages) from memory.
Stuck on a detail? Each line has a § reference to your full Unit guide (U1 §2.5 = Unit 1 guide, section 2.5).

🎯 ANSWER FORMULA FOR EVERY LONG QUESTION

1. Definition (exact textbook words) → **2. Diagram** (neat, labelled, numbered arrows) → **3. Points/steps** (one heading each, 1–2 lines) → **4. Example** (weather data, Amazon, bank…) → **5. One-line conclusion**.
A clean diagram alone is often worth 1–2 marks.

🔥 Most likely CIE-1 questions

Unit 1

1. Classify digital data with examples + comparison table.
2. Techniques to deal with unstructured data.
3. Define Big Data (Gartner) + 3 Vs + other Vs.
4. Challenges with Big Data / top challenges facing Big Data.
5. Traditional BI vs Big Data.
6. What is BDA? Classification (Analytics 1.0/2.0/3.0).
7. Data science, data scientist skills & responsibilities.
8. SMP vs MPP; shared-nothing; CAP theorem.

Unit 2

1. MapReduce for max temperature (data flow + code).
2. Design of HDFS; blocks; NameNode/DataNode.
3. Anatomy of a file read (6 steps).
4. Anatomy of a file write (7 steps).
5. HDFS Federation & High Availability.
6. Job run in classic MapReduce (MR1) vs YARN.
7. Failures in MR1 & YARN; Fair vs Capacity scheduler.
8. Shuffle and sort.

Unit 3 (CIE-1 part)

1. Spark architecture with a neat diagram.
2. Spark components (stack).
3. SparkContext; job, stage, task.
4. Internals of job execution / run-time flow (10 steps).
5. Cluster manager types + client vs cluster mode.
6. Spark vs Hadoop MapReduce.
7. Installing Spark & first Spark program.

🔢 Numbers to remember

Fact	Value	Fact	Value
Unstructured share of enterprise data	**80–90%** (Gartner 80%)	HDFS block size (default)	**128 MB** (old: 64 MB)
Digital universe 2006 → 2011	**0.18 ZB → 1.8 ZB** (10×)	Replication factor	**3**
1 ZB	**10^{21} B** = 1 billion TB	Disk block / FS block	512 B / few KB
Disk 1990 / ~20 yrs later	1,370 MB @ 4.4 MB/s (5 min) / 1 TB @ 100 MB/s (>2.5 h)	NameNode cold start	**30+ min**
1 TB over 100 disks	**< 2 min**	Map-side buffer / spill threshold	**100 MB / 80%**
Hadoop market share	**~90%**	Reduce copier threads	**5**
Hadoop born / Apache top-level	**Feb 2006** / Jan 2008	Task attempts before failing	**4**
1 TB sort record	**209 s on 900 nodes** (Apr 2008)	NodeManager heartbeat timeout	**10 min**
Yahoo! cluster (Mar 2009)	**24,000 nodes**	Reduce slow start	**5%** maps done
MR1 scalability limit	**~4,000 nodes**	Spark speed vs MapReduce	up to **100×** (memory), ~10× (disk)
Temperature format (NCDC)	°C × 10 (−0128 = −12.8 °C); 9999 = missing	Spark Web UI / Standalone master	port **4040** / **7077** (UI 8080)

Unit 1 Types of Digital Data · Introduction to Big Data · Big Data Analytics — Seema Acharya & Subhashini C

Ch 1 · Types of Digital Data

Data (U1 §1.1)

Data = quantities, characters or symbols on which operations are performed by a computer; stored & transmitted as electrical signals; recorded on magnetic, optical or mechanical media. **Data → Information → Insights.** Types: **Quantitative** (discrete = counted, continuous = measured) vs **Qualitative** (descriptive). Sources: internal & external (outside the firewall); homogeneous & heterogeneous.

3 types of digital data (U1 §1.2)

	Structured ~10%	Semi ~10%	Unstructured 80–90%
Meaning	Rows & columns; easily used by programs	No data model, but has some structure (tags); self-describing	No data model; not easily used by programs
Examples	RDBMS, Excel, MS Access	XML, JSON, HTML, email	Images, video, chats, PPTs, body of email

Structured data (U1 §1.3)

- Stored in **RDBMS** (relational model). **Cardinality** = no. of rows; **degree** = no. of columns.
- Design: fields + data types → constraints (**UNIQUE, NOT NULL**, business rule e.g. ≥ 50, permissible values e.g. CS/IS/MS) → relate tables by common column (PRIMARY KEY / FK).
- Sources: Oracle, DB2, SQL Server, Teradata, MySQL, PostgreSQL, MS Access, Excel, OLTP.
- **Ease of working (ISIST): I**nsert/update/delete (DML) · **S**ecurity (encryption, tokenization) · **I**ndexing (faster SELECT) · **S**calability (scale up the server) · **T**ransactions (**ACID**).
- **ACID:** Atomicity (all or nothing) · Consistency (valid state → valid state) · Isolation (as if alone) · Durability (permanent).

Semi-structured data (U1 §1.4)

- **5 features:** not a relational data model · **tags** segregate semantic elements · tags enforce **hierarchies** · **no separation of data & schema** · same-class entities may have different attributes / order.
- **4 characteristics:** inconsistent structure · self-describing (label/value pairs) · schema blended with data · attributes not known beforehand.
- **XML ↔ SOAP · JSON ↔ REST** · MongoDB & Couchbase/CouchDB store **JSON** natively.

Unstructured data (U1 §1.5–1.6)

- No data model; can't be stored in rows/columns; difficult to search; e.g. Aadhaar images, CCTV, social media.

- **Machine-generated:** satellite images, atmospheric data, radar, CCTV, medical imaging. **Human-generated:** emails, office docs, text messages, audio, social media.
- **Terminology issues (3):** structure may be implied · structure not helpful for the task · structure unanticipated.
- **7 techniques: Data mining** (= KDD; association rules — bread & butter, regression — marks from study hours, collaborative filtering — recommendations) · **Text analytics** (categorisation, clustering, sentiment, entity extraction) · **NLP** (understand human language) · **Noisy text analytics** ("u coming tmrw??") · **Manual tagging with metadata** · **POS tagging** (noun/verb...) · **UIMA** (open-source, by IBM, content analytics).

Comparison — 7 properties (U1 §1.7)

	Structured	Semi	Unstructured
Technology	Relational tables	XML / RDF	Character & binary
Transactions	Matured, concurrency	Adapted, not matured	None
Versioning	Tuples/rows/tables	Tuples or graph	As a whole
Flexibility	Schema-dependent, least	Medium	Most (no schema)
Scalability	Very difficult	Simpler	Most scalable
Robustness	Very robust	New, less widespread	—
Query	Complex joins	Anonymous nodes	Only textual

🧠 **CH 1 TRICKS**

ISIST (5 eases) · "**D**ark **T**ea **N**ever **N**eeds **M**ore **P**ure **U**pgrades" (7 techniques) · "**T**he **T**iny **V**ampire **F**lew **S**wiftly, **R**arely **Q**uerying" (7 properties). Structured → Unstructured: flexibility & scalability ↑, robustness/transactions/query ↓.

Ch 2 · Introduction to Big Data

Characteristics & evolution (U1 §2.1–2.2)

- **3 Cs: Composition** (structure, sources, granularity, types, static/streaming) · **Condition** (usable as is or needs cleansing?) · **Context** (where/why generated).
- **Evolution:** ≤1970s **mainframes** — primitive & structured → 1980s–90s **RDBMS** — complex & relational → 2000s+ **WWW & IoT** — complex & unstructured (data explosion).

Definition (U1 §2.3)

GARTNER

"Big Data is **high-volume, high-velocity and high-variety** information assets that demand **cost-effective, innovative forms of information processing** for **enhanced insight and decision making**."

Data → Information → Actionable intelligence → Better decisions → Enhanced business value. Informal: beyond human/technical infrastructure · today's BIG is tomorrow's NORMAL · TB/PB/ZB · 3 Vs.

Challenges with Big Data (U1 §2.4)

6: huge **growth** (useful vs noise) · **cloud** & virtualisation · data **retention** (how long) · **lack of skilled** people · **capture/storage/processing/security** · **visualisation** (+ shortage of experts). Figure: capture, storage, curation, search, analysis, transfer, visualisation, privacy.

3 Vs + others (U1 §2.5–2.6)

- **Volume** — KB→MB→GB→TB→PB→EB→ZB→YB (×1024). Sources: **internal** (file systems, SQL, NoSQL, archives) · **external** (public web: Wikipedia, weather, census) · **both** (sensors, machine logs, social media, business apps, media, docs).
- **Velocity** — batch → periodic → near real time → real time.
- **Variety** — structured + semi-structured + unstructured.
- Not definitional: **Veracity** (trustworthiness: noise, bias) & **Validity** (accuracy for the problem) · **Volatility** (how long data stays valid) · **Variability** (inconsistent flow, sudden peaks).

Why Big Data · BI vs Big Data (U1 §2.7–2.8)

More data → more accurate analysis → more confidence → operational efficiency (cost ↓, time ↓, new products, smart decisions).

	Traditional BI	Big Data
Storage	Central server	Distributed file system
Scaling	Vertical (scale up)	Horizontal (scale out)
Mode	Offline	Real-time + offline
Data / movement	Structured; data → processing	Variety; processing → data
Schema	Schema-on-write	Schema-on-read

🧠 **CH 2 TRICKS**

3 Cs · river = 3 Vs (how much, how fast, how many kinds) · "**G**iant **C**louds **R**arely **L**ack **S**ome **V**iews" (6 challenges) · "**K**ind **M**en **G**ive **T**ea, **P**lease **E**at **Z**esty **Y**ogurt" (units).

Ch 3 · Big Data Analytics

What BDA is (U1 §3.1–3.2)

- Examples: **Amazon recommendations**; **500-truck** urgent delivery. Large data → analyse → patterns → better decisions.
- Pyramid: Data (collect) → Information (organise) → Knowledge (summarise, analyse) → Actionable insights (synthesise) → Decision.
- Data from: websites, billing (POS), ERP, CRM, RFID, social media.
- **Is (6):** technology-enabled · deeper insights · competitive edge · handshake of **IT + data scientists + business users** · large & varied data · **moving code to data**.
- **Isn't (6):** only volume · just technology · replacement for RDBMS · replacement for data warehouse · only for Google/Amazon · one-size-fits-all RDBMS.
- **Hype (3):** data growth · cheaper storage · tools available.

Classification (U1 §3.3)

School 1 (BOAM): Basic (reports, slicing) · Operationalised (in business processes) · Advanced (predictive/prescriptive) ·

Monetised (direct revenue).

	1.0 (1950s–2009)	2.0 (2005–2012)	3.0 (2012–)
Type	Descriptive	+ Predictive	+ Prescriptive
Question	What happened?	What will happen?	What should we do?
Data	Small, structured, internal (DW)	Big, unstructured, external (Hadoop)	Blend, internal + external
Tech	RDBMS	Hadoop, appliances	In-memory, in-DB, ML

Descriptive → Diagnostic → Predictive → Prescriptive = hindsight → insight → foresight.

Top challenges · importance (U1 §3.4–3.5)

- **7 top challenges:** Scale · Security · Schema (dynamic) · Continuous availability (24/7) · Consistency · Partition tolerance · Data quality.
- **4 approaches:** Reactive–BI · Reactive–BDA (huge but static) · Proactive–analytics (traditional DB limits) · **Proactive–BDA ★** (TB/PB/EB, high-performance).

Data science & data scientist (U1 §3.6–3.7)

- **Data science** = science of extracting knowledge from data (maths, statistics, IT, ML, pattern recognition). Uses: weather, oil drilling, seismic, fraud, social media, retail.
- **Skills (BTM): Business acumen** (domain, strategy, problem solving, communication, presentation, inquisitiveness) · **Technology** (RDBMS, NoSQL, programming, DW, data mining, visualisation) · **Mathematics** (maths, statistics, algorithms, ML, pattern recognition, NLP).
- Process: collect → prepare → analyse → visualise → communicate → decide.
- **4 responsibilities:** data management · analytical techniques · business/domain expertise · communication & presentation.

Terminologies (U1 §3.8–3.9)

- **In-memory analytics:** keep relevant data in **RAM** → faster, rapid deployment, better insights, minimal IT.
- **In-database processing:** OLTP → ETL → EDW; run computations **inside the DB** → no export.
- **SMP:** shared main memory, **one OS**, tightly coupled, caches + system bus. **MPP:** each processor own memory + OS, messaging, harder to program.
- **Parallel:** tightly coupled, user unaware. **Distributed:** loosely coupled machines, own users, data spread.
- **Shared memory / shared disk / shared nothing.** SN advantages: **fault isolation** + **scalability**.
- **CAP (Brewer):** Consistency (read = last write) · Availability (always a response) · Partition tolerance (works despite network break) → pick **2**. **CA**: RDBMS (MySQL, PostgreSQL) · **CP**: HBase, MongoDB, Redis, MemcacheDB, BigTable · **AP**: Riak, Cassandra, CouchDB, Dynamo. Prefer C for bank balance; A for feeds/caches.

🧠 **CH 3 TRICKS**

"**T**he **D**eep **C**ompetitive **H**andshake **W**ith **M**ovement" (6 is) · **BOAM** · "**D**o **D**octors **P**rescribe **P**roperly?" · "**S**ix **S**tudents **S**at **C**almly, **C**opying **P**ast **Q**uizzes" (7 challenges) · **BTM** · "**M**y **A**unt **B**akes **C**akes" (4 roles) · SMP "Same Memory, Please" / MPP "My Private Partition" · CP "**H**appy **M**onkeys **R**un **M**any **B**ananas" · AP "**R**eal **C**ats **C**hase **D**ogs".

Unit 2 Data Analytics with Hadoop — Tom White, *Hadoop: The Definitive Guide*

Ch 1 · Basics of Hadoop

Data! · storage & analysis (U2 §1.1–1.2)

- NYSE 1 TB/day · Facebook 10 bn photos ≈ 1 PB · Ancestry 2.5 PB · Internet Archive 2 PB (+20 TB/month) · LHC 15 PB/yr · MyLifeBits ≈ 1 GB/month. Machines (logs, RFID, sensors, GPS) generate even more.
- **Problem:** disk **capacity** grew much faster than **transfer speed** → read from **100 disks in parallel**.
- New problems: **hardware failure** → solved by **replication** (HDFS) · **combining data** → solved by **MapReduce**.

Hadoop (U2 §1.3)

DEFINITION

Open-source framework to **store and process large data sets in a parallel and distributed manner**. Hadoop = reliable shared storage + reliable distributed analysis.

- Written in **Java**; by **Doug Cutting & Mike Cafarella**; named after a **toy elephant** (not an acronym).
- **Components:** HDFS (storage) · MapReduce (processing) · YARN (resource management) · Common (utilities).
- History: Nutch 2002 → Google GFS 2003 → NDFS 2004 → MapReduce paper 2004 → Nutch + MR 2005 → **Hadoop Feb 2006**. Yahoo! scaled it (24,000 nodes by 2009).
- **Why:** ~90% market, scalable, fault tolerant, flexible with data.
- Ecosystem: Hive, Pig, Mahout, Spark (SQL, Streaming, MLlib, GraphX), HBase, Cassandra, ZooKeeper, Flume, Kafka, Sqoop, Storm.

NCDC weather data (U2 §1.4)

Semi-structured, record-oriented, fixed-width ASCII line per observation: station ID, date, time, … **air temperature (°C × 10)**, pressure. 9999 = missing. Many small gzipped station files → **preprocess** into one file per year.

Map & Reduce (U2 §1.5–1.6)

- **Two phases**, each with **key–value** input/output; programmer writes **map** and **reduce** functions.
- **Map** = extract (year, temp), filter bad records · **Shuffle & sort** = group by key: 1949 → [111, 78] · **Reduce** = max → (1949, 111), (1950, 22).
- `Mapper<LongWritable, Text, Text, IntWritable>` = (offset, line) → (year, temp). `Reducer<Text, IntWritable, Text, IntWritable>`.

- **Driver:** Job, setJarByClass, input/output paths (output dir must not exist), setMapperClass, setReducerClass, output key/value classes, `waitForCompletion`. Output: `part-r-00000`.
- Test run on 5 records: `hadoop MaxTemperature input/ncdc/sample.txt output`.

Scaling out & data flow (U2 §1.7)

- **Job** = input + program + config. **Tasks** = map + reduce, scheduled by **YARN**; failed tasks rescheduled.
- **Input split** → one map task per split; small splits = better load balancing.
- **Data locality:** data-local → rack-local → off-rack. **Optimal split = block size** (largest input guaranteed on one node).
- Map output → **local disk** (intermediate); reduce output → **HDFS**.
- Multiple reducers: map output **partitioned** (one partition per reducer, default **hash partitioner**) → "**the shuffle**". Zero reducers possible. **Combiner** = mini-reducer on map output.

Ch 2 · HDFS

Design (U2 §2.1–2.2)

DEFINITION

HDFS is a file system designed for storing **very large files** with **streaming data access patterns**, running on clusters of **commodity hardware**.

- **Good at (LSC):** very large files (MB→PB) · streaming = **write once, read many** · cheap commodity hardware (keeps working when nodes fail).
- **Bad at (LSM): low-latency** access (use **HBase**) · **lots of small files** (NameNode memory: millions OK, billions not) · **multiple writers / arbitrary modifications** (single writer, append at end).

Blocks (U2 §2.3)

128 MB default; a 1 MB file uses 1 MB. **Benefits:** file can be larger than any disk · simpler storage subsystem · fits **replication** (3 copies on separate machines).

NameNode & DataNodes (U2 §2.4)

- **Master–worker. NameNode**: namespace, file tree, metadata — persisted as **namespace image + edit log**; block locations rebuilt from DataNode reports. **DataNodes**: store/retrieve blocks, send **block reports**.
- Lose the NameNode = lose the file system. Protect it by: (1) writing metadata to **multiple file systems** (local + **NFS**, synchronous & atomic); (2) **Secondary NameNode** — periodically **merges image + edit log** (checkpoint); **not** a backup.

Federation & HA (U2 §2.5–2.6)

- **Federation** (2.x): NameNode memory limits scaling → **add NameNodes**, each owns part of the namespace (/user, /share). **Namespace volume** = metadata + **block pool**; independent; one failing doesn't affect others.
- **HA:** NameNode = **SPOF**. Cold start needs (i) load image (ii) replay edit log (iii) block reports → leave safe mode = **30+ min**. Fix: **active–standby** pair: shared edit log (HA storage) · DataNodes report to **both** · client failover · standby does checkpoints.

File read — 6 steps (U2 §2.7)

1. `open()` on **DistributedFileSystem**.
2. RPC to **NameNode** → block locations, DataNodes **sorted by proximity**; returns **FSDataInputStream** (wraps DFSInputStream).
3. `read()` → connect to **closest DataNode** for block 1.
4. Data **streamed** to client.
5. End of block → close, find best DataNode for **next block** (asks NN for next batch).
6. `close()`.

Failure: try next closest DN, remember bad DNs · **checksum** fails → read another replica + report corrupt block to NN.

File write — 7 steps (U2 §2.8)

1. `create()` on DistributedFileSystem.
2. RPC to NN to **create** file (no blocks); checks **exists? permission?** else **IOException**; returns **FSDataOutputStream**.
3. `write()` → **packets** → **data queue** → **DataStreamer** asks NN for blocks → **pipeline** of 3 DNs.
4. **Write packet**: DN1 stores & forwards → DN2 → DN3.
5. **Ack packet** back; removed from **ack queue** when all DNs ack.
6. `close()` flushes remaining packets, waits for acks.
7. Tell NN file **complete**.

Failure: close pipeline · ack-queue packets to front of data queue · good block gets **new identity** (partial one deleted later) · remove failed DN, new pipeline from 2 good DNs.

🧠 HDFS TRICKS

LSC strengths / **LSM** weaknesses · library: NameNode = catalogue, DataNodes = shelves · read "**O**nly **G**ood **R**eaders **R**eally **R**ead **C**arefully" · write "**C**ute **C**ats **W**rite **P**oems **A**nd **C**lose **C**arefully".

Ch 3 · Developing a MapReduce Application

Workflows · MRUnit · job run (U2 §3.1–3.3)

- Complex problem → **more jobs**, not more complex jobs (or Pig, Hive, Cascading, Crunch, Spark). Mean-max = Job 1 (station, date → max) + Job 2 (station, day-month → mean).
- Linear chain: `JobClient.runJob()` one after another; **DAG** → **JobControl** (stores jobs, dependencies, order, monitors, skips dependents on failure).
- **MRUnit** + JUnit: **MapDriver** (record → (1950, −11)), **ReduceDriver** ([10, 5] → 10), whole job via **local job runner**.
- `submit()` / `waitForCompletion()`; `mapreduce.framework.name` = **local** (one JVM) / **classic** (MR1) / **yarn** (MR2).

Classic MapReduce (MR1) — 10 steps (U2 §3.4)

Entities: Client · **JobTracker** (coordinates) · **TaskTrackers** (run tasks) · DFS.

- **Submission 1–4:** JobSubmitter · new job ID from JobTracker · check output dir · compute splits · copy JAR (high replication), config, splits to shared FS · `submitJob()`.
- **Initialisation 5–6:** internal queue → scheduler creates job object · retrieve splits · **one map task per split**, reducers = `mapred.reduce.tasks`.
- **Assignment 7:** TaskTracker **heartbeat** → gets a task.

- **Execution 8–10:** localise JAR + distributed cache · **TaskRunner launches new JVM** · run task (user bugs can't crash TaskTracker).

YARN (MR2) — 11 steps (U2 §3.5)

Why: MR1 bottleneck at ~**4,000 nodes**; JobTracker did scheduling + monitoring → split into **ResourceManager** + per-app **ApplicationMaster**. YARN = **Yet Another Resource Negotiator** (Yahoo!, 2010). **Entities:** Client · ResourceManager · NodeManagers · MRAppMaster · DFS.

- **Submission 1–4:** application ID from RM · checks · copy resources · `submitApplication()`.
- **Init 5a/5b–7:** scheduler allocates container, RM launches **MRAppMaster** · AM initialises job · retrieves splits.
- **Assignment 8:** AM requests containers — maps first (higher priority); reduces after **5%** maps done (uber task = tiny job in AM's JVM).
- **Execution 9a/9b–11:** AM asks NodeManager to start container · **YarnChild** localises resources · runs task. Tasks report progress to the AM.

Failures (U2 §3.6)

	MR1	YARN
Task	Runtime exception / JVM exit → TaskTracker marks failed	Reported to AM; hang = no ping; failed after **4 attempts**
Worker	TaskTracker stops heartbeats → removed; **blacklisted** if >4 tasks of a job fail	NodeManager no heartbeat **10 min** → removed; AM blacklists if >3 fail
Coordinator	—	AM fails → RM restarts it in new container
Master	**JobTracker = SPOF → job fails**	RM recovers from **checkpointed state**

Scheduling (U2 §3.7)

- **FIFO** (MR1 default). **Fair**: every user a fair share over time; **pools** (one per user); single job gets whole cluster; short jobs finish; **preemption**. **Capacity**: hierarchical **queues** with fixed capacity; **FIFO within queue**; like a separate cluster per org.
- **Preemption** = forcefully take resources from one job and give to a higher-priority one.

Shuffle & sort (U2 §3.8)

- Guarantee: **reducer input sorted by key**; sort + transfer of map output = **shuffle** ("heart of MapReduce").
- **Map side: 100 MB circular buffer** → spill at **80%** (round-robin to mapred.local.dir) → **partition** → **in-memory sort** → **combiner** → merge spills into one sorted, partitioned file → (compress) → served over **HTTP**.
- **Reduce side: copy phase** (5 threads, starts as each map finishes) → **merge ("sort") phase** in rounds → **reduce phase** per key → output to HDFS.

🧠 **CH 3 TRICKS**

Phases "**S**tudents **I**n **A**n **E**xam" (Submission, Init, Assignment, Execution) · MR1 "**C**lient **J**ust **T**alks to **D**FS" · YARN "**C**an **R**ahul **N**ot **A**ttend **D**aily?" · failures MR1 "**T**wo **T**ired **J**ugglers", YARN "**T**he **A**nts **N**ever **R**est" · Fair = Pools, Capacity = Queues · map side "**B**ig **S**pills **P**lease **S**weep, **C**lean, **M**op" · reduce side Copy → Merge → Reduce.

Unit 3 · CIE-1 part

Apache Spark Overview: Architecture & Installation — Abbasi, *Learning Apache Spark 2*

Spark architecture

What is Spark (U3 §1.1)

DEFINITION

Open-source **cluster computing framework for data analytics**, designed for quick computation; supports **in-memory cluster computing** and parallel processing; faster than Hadoop.

- Processes large datasets quickly on **one or many computers**; APIs for **Scala, Java, Python** (+ SQL, R); lab = **PySpark on Ubuntu**.
- UC Berkeley AMPLab 2009 (Matei Zaharia); Apache top-level 2014; up to **100×** faster in memory, ~10× on disk.

Components (U3 §1.2)

- **Spark Core** — foundation / execution platform (APIs: R, SQL, Python, Scala, Java).
- **Spark SQL** — SQL & HiveQL on structured/semi-structured data; DataFrames.
- **Spark Streaming** — live data as **micro-batches**.
- **MLlib** — machine learning, in-memory for iterative algorithms.
- **GraphX** — graph computation.
- **SparkR** — R front end from the R shell.

Terminology (U3 §1.3)

- **SparkContext** = **entry point**; connects to the execution environment; creates RDDs, accumulators, broadcast variables; runs jobs. **Responsibilities (SCRAD):** status · cancel job/stage · run sync/async · access/unpersist cached RDD · dynamic allocation. Spark 2.0+: **SparkSession** wraps it.
- **Spark Shell** — Scala command line with auto-completion. **Spark application** — self-contained computation running user code.
- **Task** = smallest unit, one executor, **one per partition** · **Job** = parallel computation of many tasks, triggered by an **action** ·

Stage = set of dependent tasks; job split where steps must wait (shuffle). Application ⊃ Job ⊃ Stage ⊃ Task.

Architecture (U3 §1.4–1.5)

- **Master–slave:** one **driver** (central coordinator) + many **executors**; each its own **Java process**; app = driver + its executors. Cluster manager: own **standalone**, or **YARN**, **Mesos**.
- **Driver:** runs **main()**, creates **SparkContext**; job scheduling; negotiates with cluster manager; stores RDD info; client or cluster mode.
- **Executor:** process on a worker node; **runs tasks**; keeps data in memory/disk (cache); reads/writes external data; each app has its own.
- **Cluster manager: allocates resources** across apps (YARN, Mesos, Standalone) — does **not** schedule tasks.
- **Worker node:** slave node running app code. **Task:** unit of work for one executor.

Job execution & run-time flow (U3 §1.6–1.7)

Action → **job** → **DAG Scheduler** (builds operator DAG, splits into **stages**) → **Task Scheduler** (launches tasks via cluster manager) → executors on worker nodes.

1. Driver **negotiates** resources with cluster manager.
2. Cluster manager **launches executors**.
3. Executors **register** with driver.
4. Code → **logical DAG**.
5. DAG **optimised**.
6. **Physical plan** = set of stages.
7. **Tasks** sent to cluster.
8. Executors **execute**; driver **monitors**.
9. Driver **schedules** future tasks by **cached-data location**.
0. `stop()` → **terminate executors, release resources**.

Spark vs Hadoop MapReduce (U3 §1.8)

	MapReduce	Spark
Processing	Disk-based	**In-memory**
Plan	Fixed map → reduce	Optimised **DAG** of stages
Workloads	Batch	Batch, iterative, interactive, streaming
Languages	Java	Scala, Java, Python, R, SQL
Libraries	Separate (Hive, Pig...)	Built in (SQL, Streaming, MLlib, GraphX)
Storage	HDFS	None of its own (HDFS, S3, local...)

Installation & running

Install on Ubuntu (U3 §2.1)

Java (JDK) → download pre-built **.tgz** → `tar -xzf` → move to /opt/spark → set `SPARK_HOME`, `PATH` (bin, sbin) → verify `pyspark` / `spark-shell --version`. Or `pip install pyspark`. Folders: **bin** (shells, spark-submit, run-example), **sbin** (start-master/worker), conf, examples, jars, data. Shells: spark-shell, pyspark, sparkR, spark-sql (`sc`, `spark` ready-made). Web UI **localhost:4040**.

First program (U3 §2.2)

```
lines = sc.textFile("README.md")       # RDD
lines.count(); lines.first()           # actions → jobs
lines.filter(lambda l: "Spark" in l).count()
# word count app
counts = (sc.textFile("input.txt")
  .flatMap(lambda l: l.split())
  .map(lambda w: (w, 1))
  .reduceByKey(lambda a, b: a + b))
counts.collect(); spark.stop()
$ spark-submit --master local[4] wordcount.py
```

Cluster managers & modes (U3 §2.3)

Type	--master	Note
Local	local, local[K], local[*]	one machine; learning/testing
Standalone	spark://HOST:7077	Spark's own; master + workers; UI 8080
YARN	yarn	Hadoop 2; shares cluster, reads HDFS
Mesos	mesos://HOST:5050	general-purpose manager
Kubernetes	k8s://https://HOST:PORT	pods/containers (Spark 2.3+)

Client mode: driver on your machine (shells, debugging). **Cluster mode:** driver inside the cluster (production; on YARN inside the AM).

Running examples (U3 §2.4)

`./bin/run-example SparkPi 10` → "Pi is roughly 3.14..." · Python: `spark-submit examples/src/main/python/pi.py 10` (10 partitions = 10 tasks).

🧠 **UNIT 3 TRICKS**

"**C**ool **S**tudents **S**tream **M**ovies, **G**raphs & **R**eels" (6 components) · **SCRAD** "Spark Can Run Any Data" · "**A**ll **J**obs **S**plit into **T**asks" · restaurant: driver = head chef, cluster manager = manager, executor = cook, task = dish · run-time "**N L R — L O P T — E M S**" · managers "**L**et **S**park **Y**ell **M**ore **K**indly".

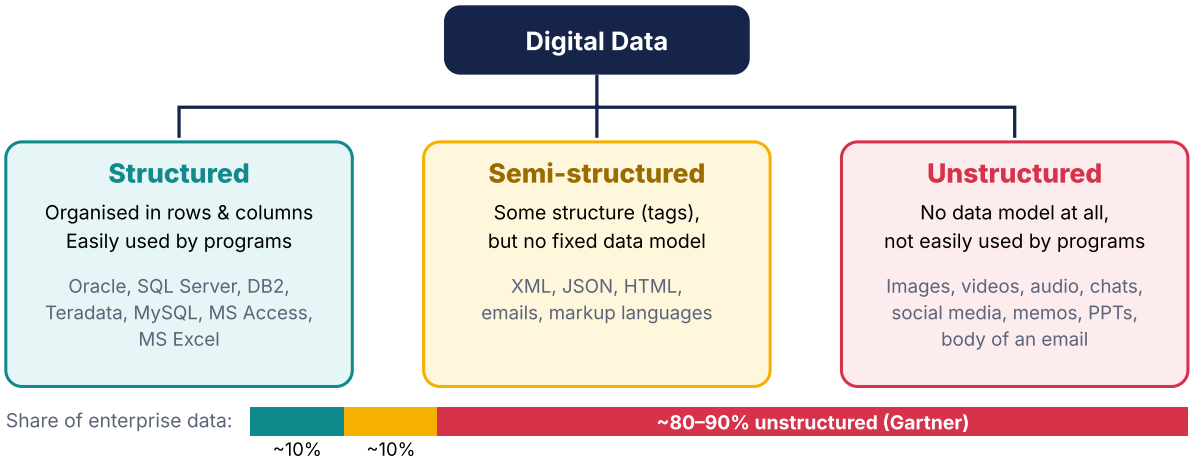
⚠️ **CLASSIC TRAPS**

Secondary NameNode ≠ backup · cluster manager ≠ task scheduler (driver schedules) · Spark ≠ storage (it uses HDFS) · Velocity ≠ Variability ≠ Volatility · "Challenges with Big Data" (6) ≠ "Top challenges facing Big Data" (7) · write **128 MB** block size.

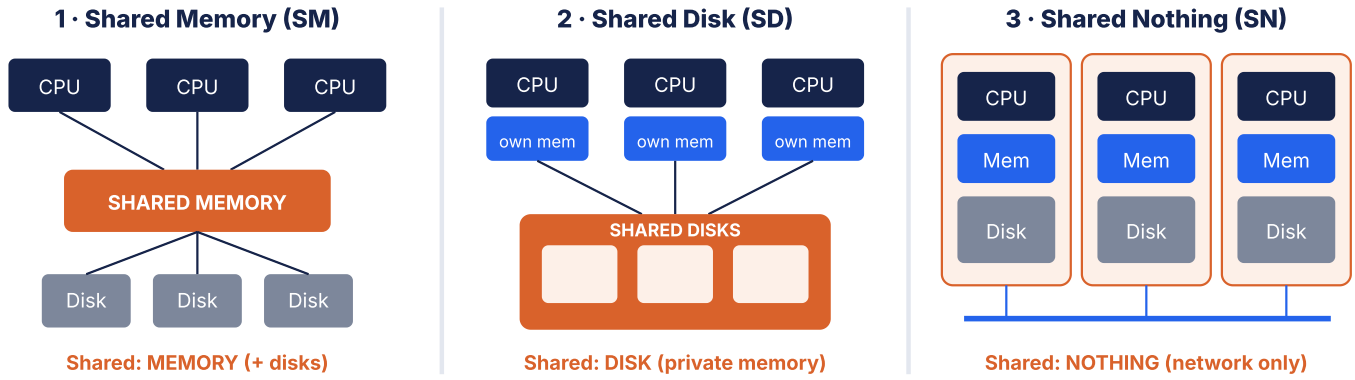
Must-draw diagrams

Look → cover → redraw from memory → check. Numbered arrows = exam steps.

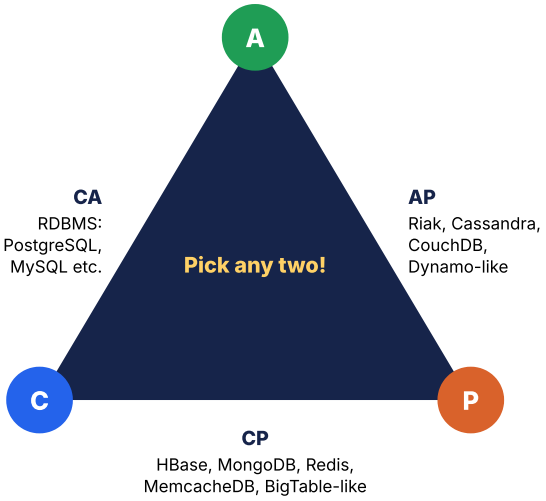
1. Unit 1 · Classification of digital data (U1 §1.2)



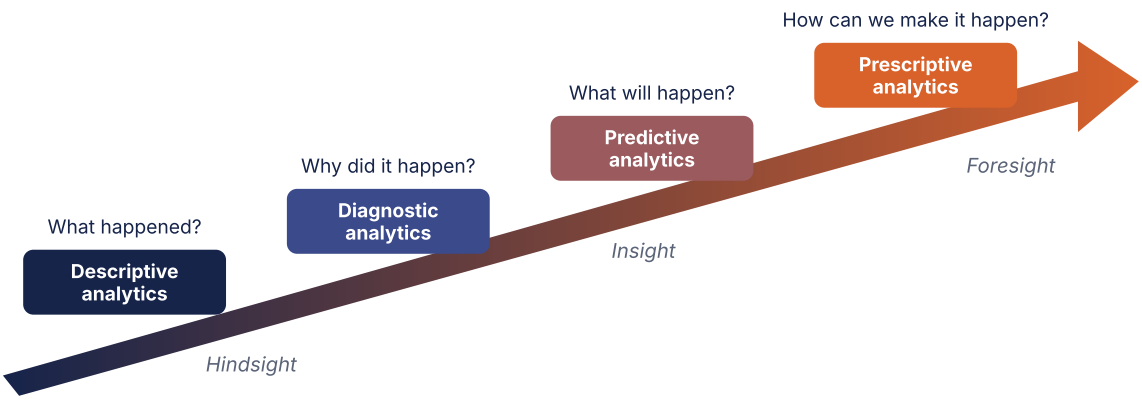
2. Unit 1 · Shared memory / shared disk / shared nothing (U1 §3.8)



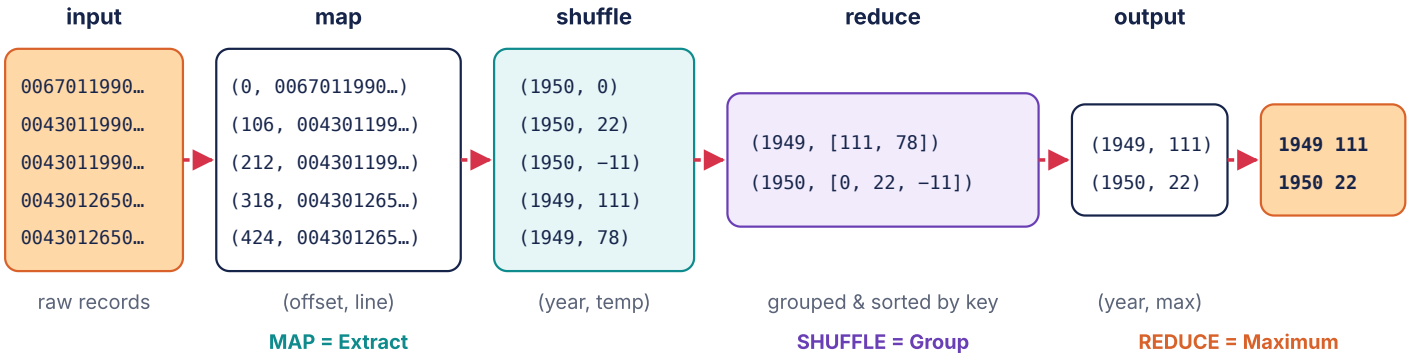
3. Unit 1 · CAP theorem with databases (U1 §3.9)



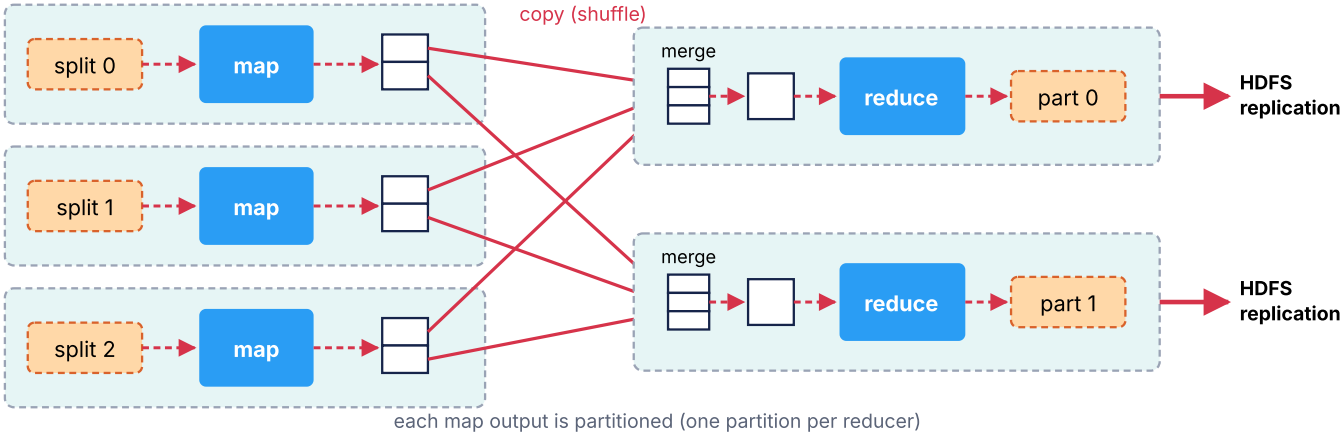
4. Unit 1 · Analytics: hindsight → insight → foresight (U1 §3.3)



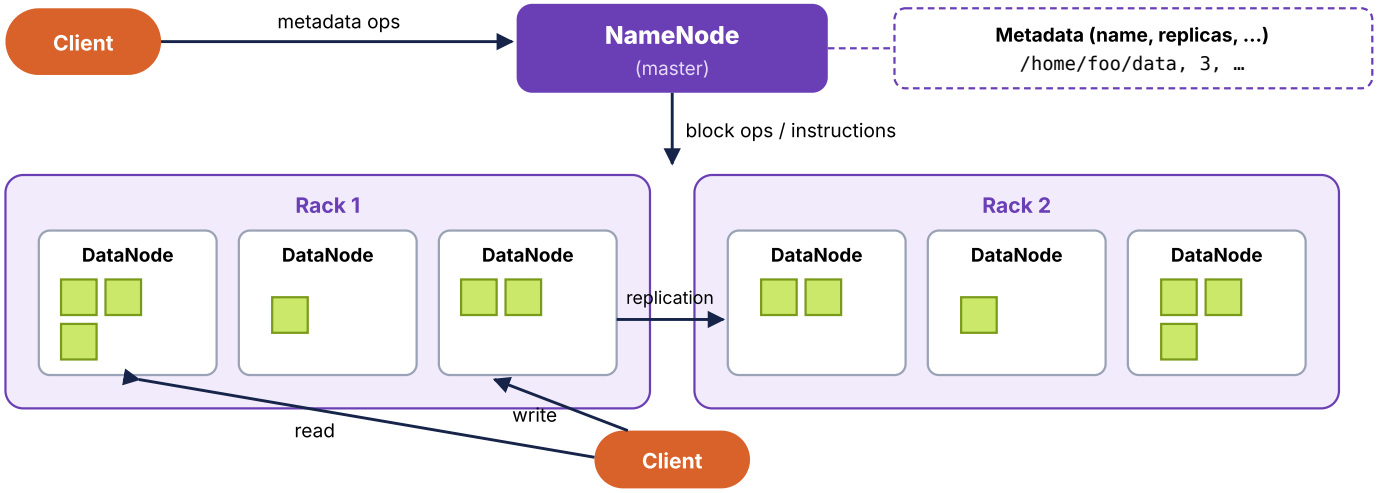
5. Unit 2 · MapReduce logical data flow (max temperature) (U2 §1.5)



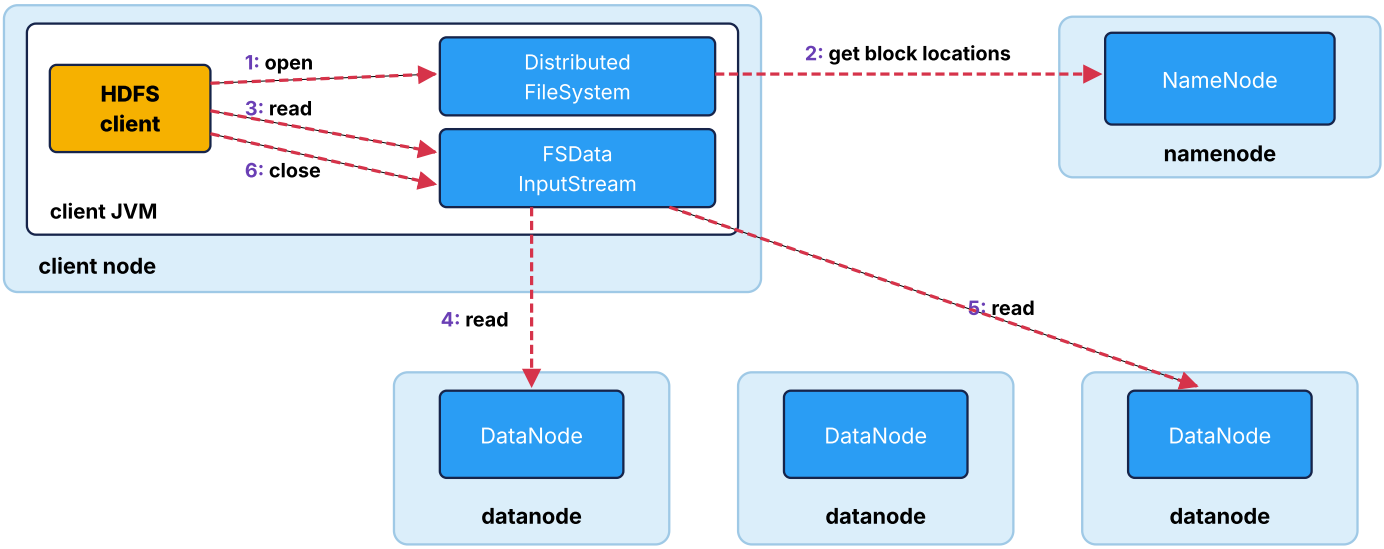
6. Unit 2 · Data flow with multiple reducers (U2 §1.7)



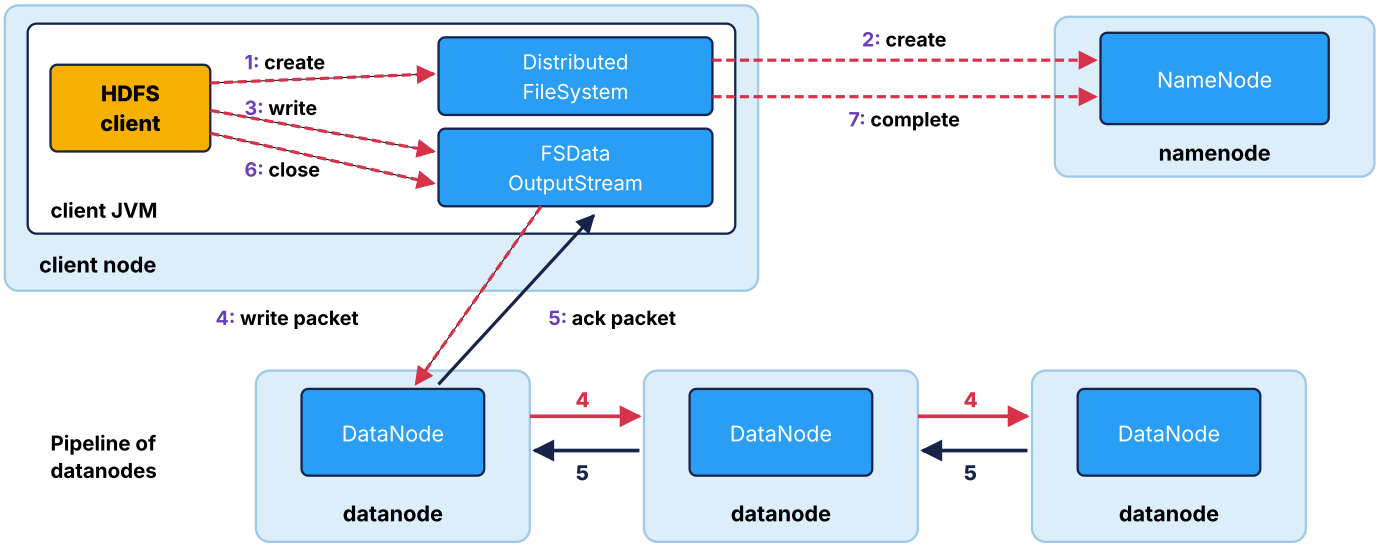
7. Unit 2 · HDFS architecture (U2 §2.4)



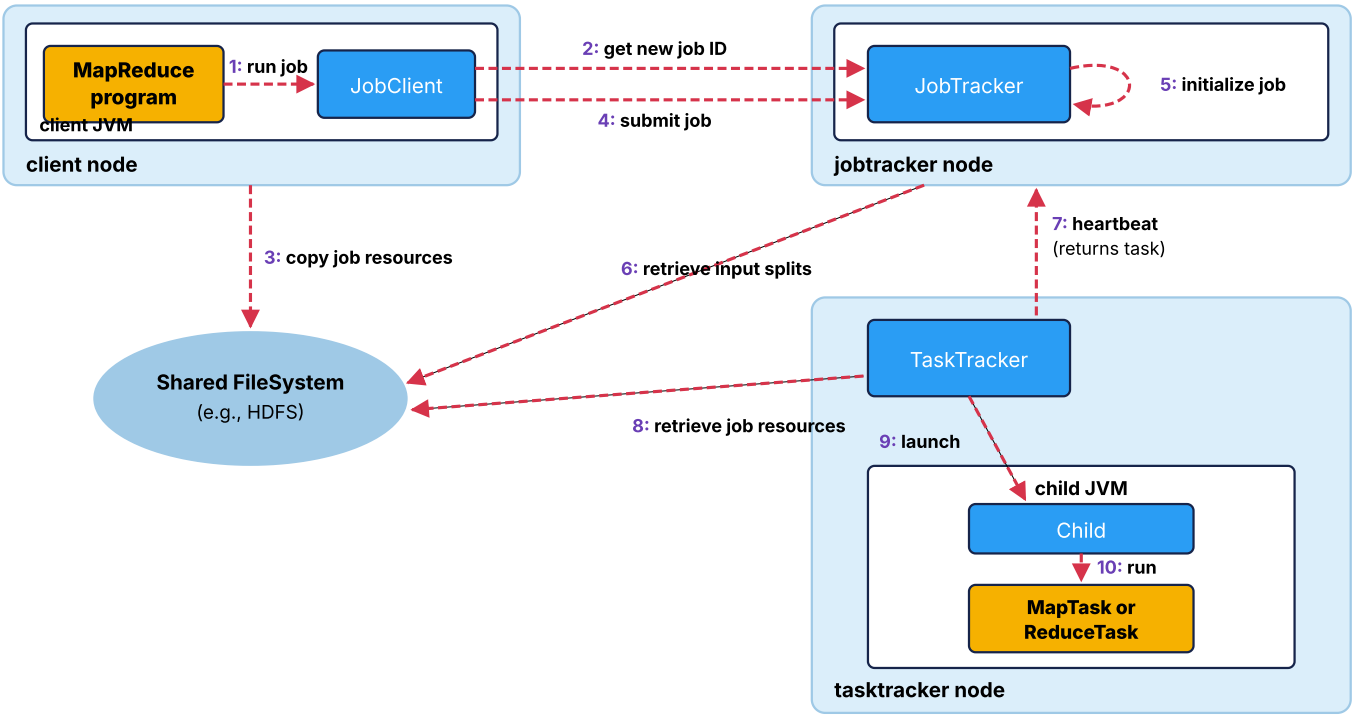
8. Unit 2 · Anatomy of a file read (steps 1–6) (U2 §2.7)



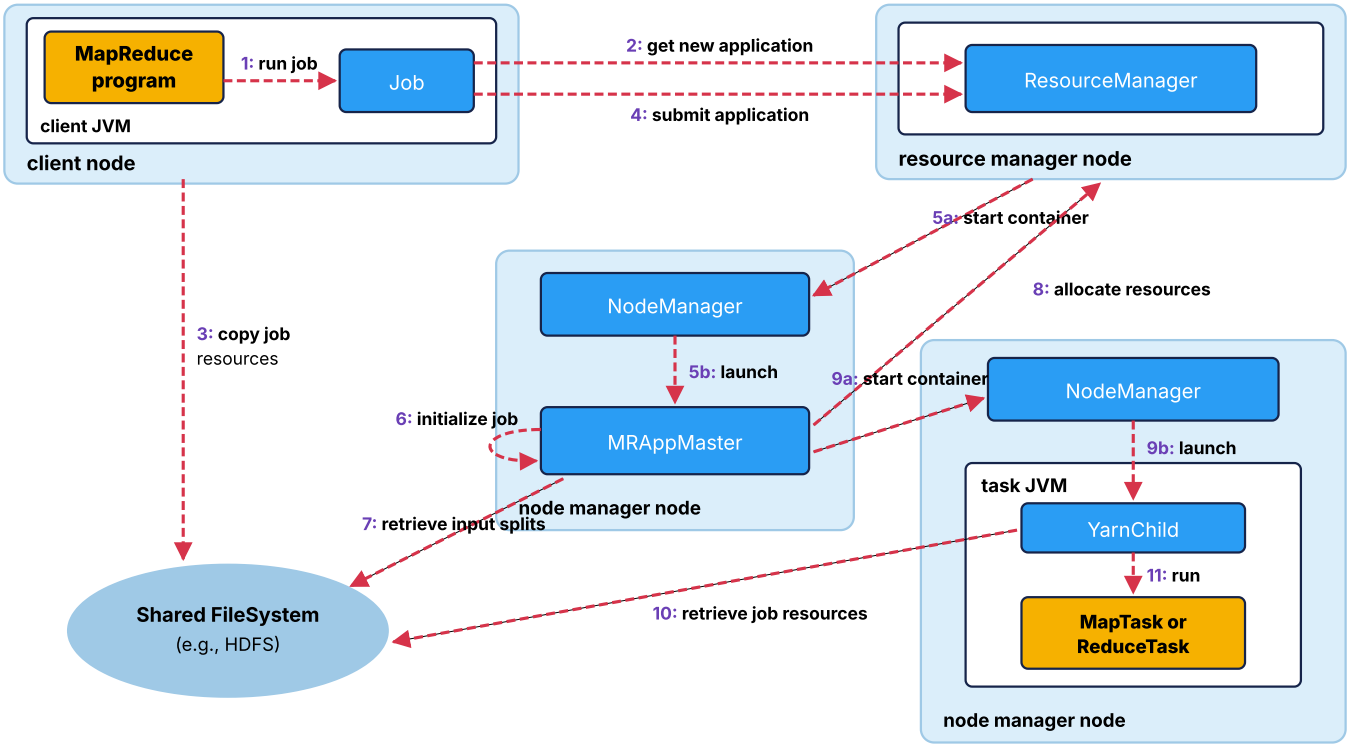
9. Unit 2 · Anatomy of a file write (steps 1–7) (U2 §2.8)



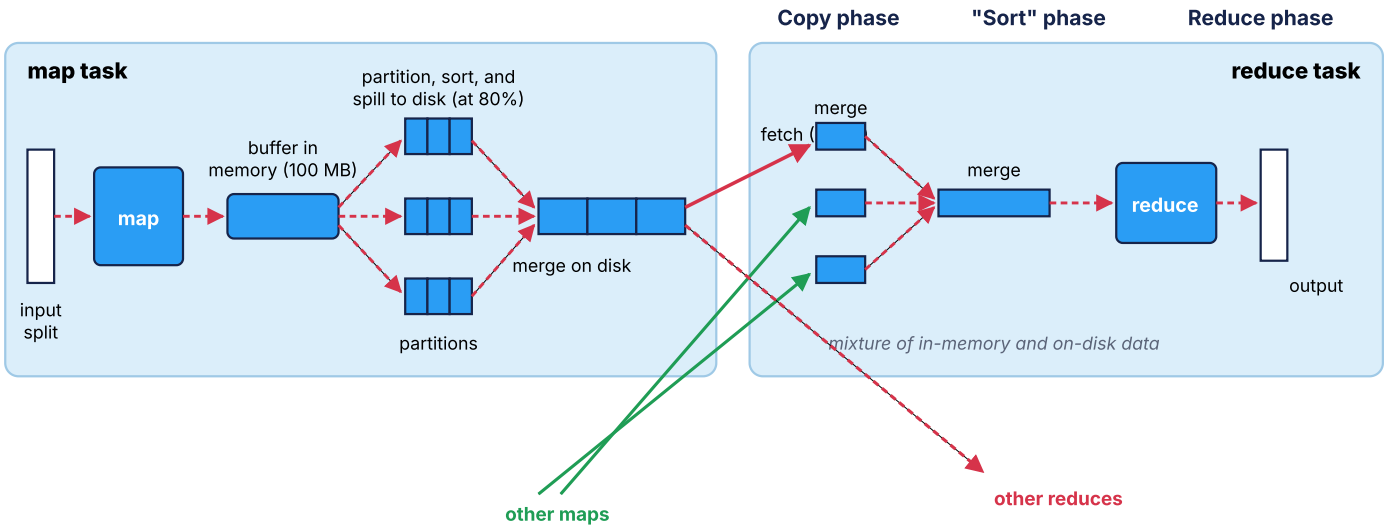
10. Unit 2 · Job run in classic MapReduce (MR1, steps 1–10) (U2 §3.4)



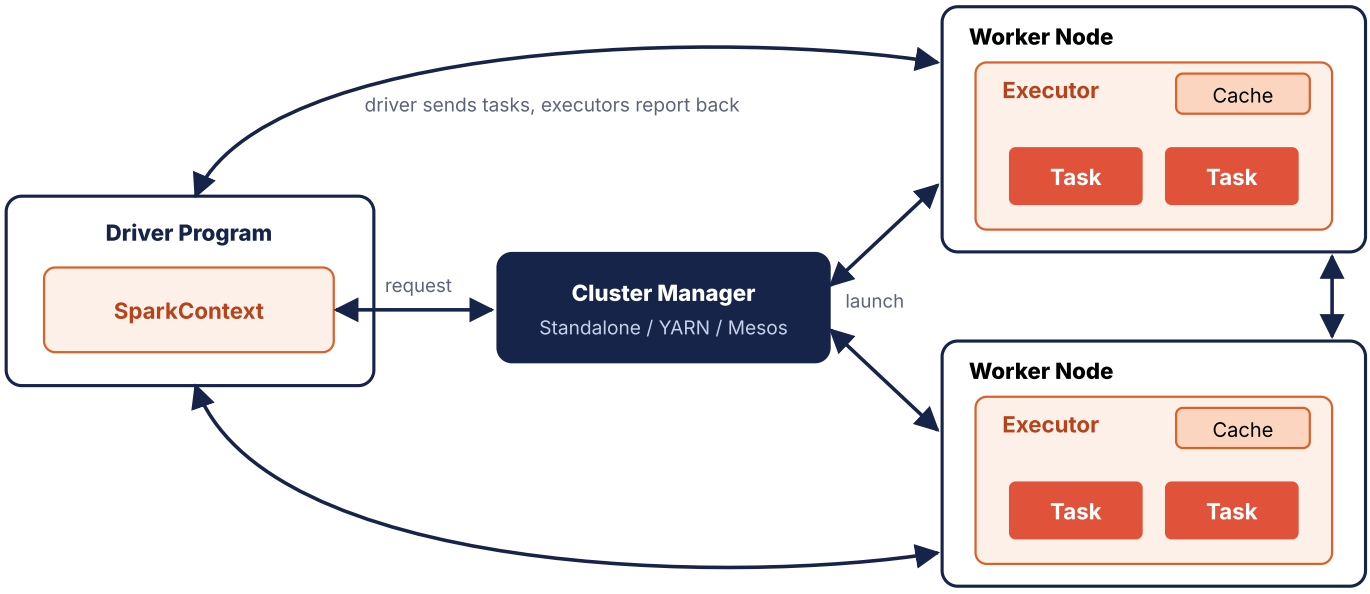
11. Unit 2 · Job run on YARN (steps 1–11) (U2 §3.5)



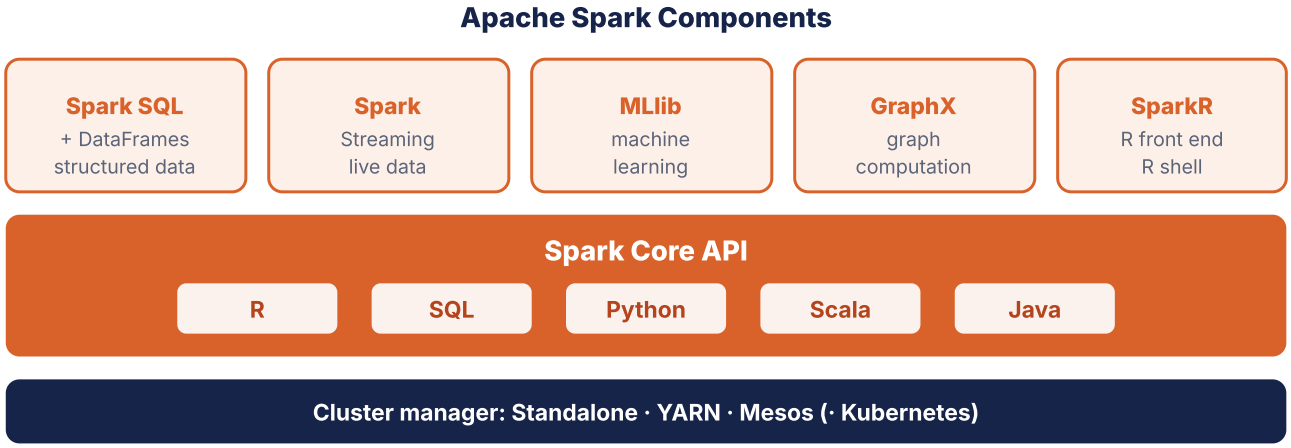
12. Unit 2 · Shuffle and sort (U2 §3.8)



13. Unit 3 · Spark architecture (U3 §1.5)



14. Unit 3 · Spark components (stack) (U3 §1.2)



15. Unit 3 · Internals of job execution (U3 §1.6)

